

Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών  
Πανεπιστήμιο Θεσσαλίας

## ECE333 - Εργαστήριο Ψηφιακών Συστημάτων

Χειμερινό Εξάμηνο — Ακαδημαϊκό Έτος 2025-2026

### Εργαστηριακή Εργασία 3<sup>η</sup>

### Υλοποίηση Ελεγκτή HDMI

High Definition Multimedia Interface (HDMI) Driver

24/11/2025 έως 19/12/2025

Δ. Γαρυφάλλου

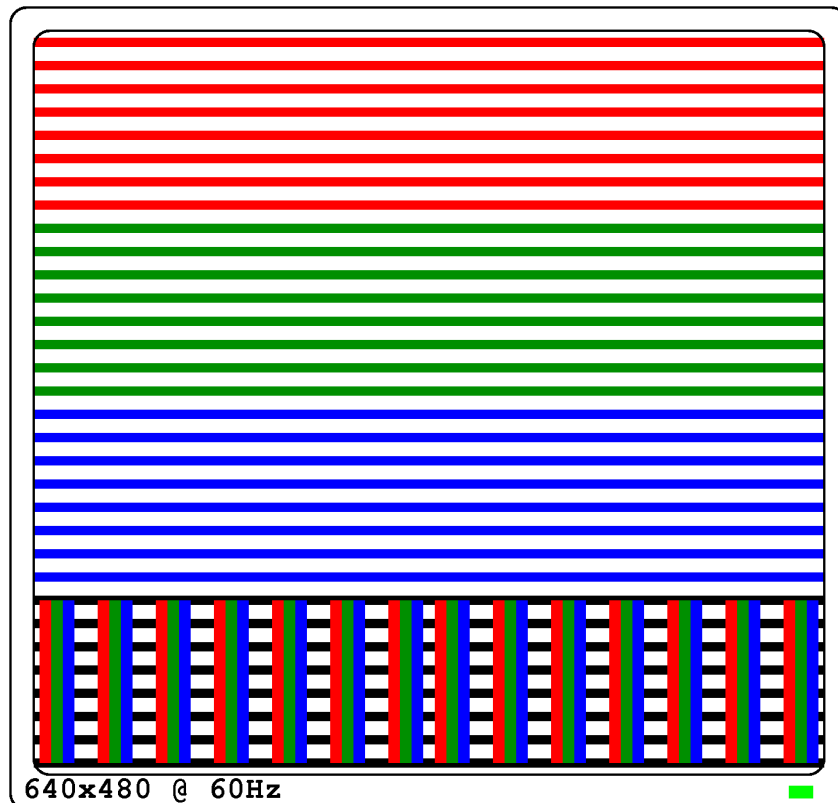
## Περιεχόμενα

|          |                                                                                                          |           |
|----------|----------------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Στόχος της 3ης Εργασίας</b>                                                                           | <b>3</b>  |
| <b>2</b> | <b>Το HDMI</b>                                                                                           | <b>4</b>  |
| 2.1      | Ψηφιακή Αντιστοίχιση Χρωμάτων . . . . .                                                                  | 4         |
| 2.2      | Χρονισμός του HDMI . . . . .                                                                             | 4         |
| 2.2.1    | Τιμές Διαστημάτων του Frame . . . . .                                                                    | 5         |
| 2.3      | Χρήση ρολογιού . . . . .                                                                                 | 7         |
| 2.4      | Video RAM (VRAM) . . . . .                                                                               | 7         |
| 2.5      | Κωδικοποίηση TMDS . . . . .                                                                              | 7         |
| 2.6      | Σειριοποίηση (Serialization) . . . . .                                                                   | 8         |
| <b>3</b> | <b>Δομή της Μονάδας Ελεγκτή/Οδηγού HDMI</b>                                                              | <b>9</b>  |
| <b>4</b> | <b>Μέρος Α - Υλοποίηση VRAM</b>                                                                          | <b>9</b>  |
| <b>5</b> | <b>Μέρος Β - Υλοποίηση Οριζόντιου και Κάθετου μετρητή Pixel και οδήγηση HSYNC, VSYNC, και ACTIVEDRAW</b> | <b>10</b> |
| <b>6</b> | <b>Μέρος Γ - Κωδικοποίηση TMDS</b>                                                                       | <b>10</b> |
| 6.1      | Ελαχιστοποίηση μεταβάσεων (Transition Minimization) . . . . .                                            | 11        |
| 6.2      | Ολοκλήρωση κωδικοποίησης TMDS . . . . .                                                                  | 12        |

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>7</b> | <b>Μέρος Δ - Ολοκλήρωση του Ελεγκτή/Οδηγού HDMI</b>          | <b>13</b> |
| 7.1      | Περιορισμοί XDC . . . . .                                    | 15        |
| 7.2      | Προχωρημένη Χρήση του Ελεγκτή HDMI (Προαιρετικό) . . . . .   | 15        |
| <b>8</b> | <b>Προθεσμία Παράδοσης, Υποβολή της Εργασίας και Αναφορά</b> | <b>16</b> |
| <b>9</b> | <b>Ερωτήσεις και Απορίες</b>                                 | <b>16</b> |

# 1 Στόχος της 3ης Εργασίας

Ο στόχος της 3<sup>ης</sup> εργαστηριακής εργασίας είναι η υλοποίηση ενός Ελεγκτή/Οδηγού οθόνης HDMI (High Definition Multimedia Interface), για να επιτευχθεί η οδήγηση μιας συμβατικής οθόνης και η εμφάνιση εικόνας σε αυτήν. Μέρος της εσωτερικής μνήμης RAM της FPGA, θα πρέπει να οριστεί σαν Μνήμη Εικόνας/Όρασης - Video RAM (VRAM) του οδηγού, έτσι ώστε ο οδηγός να την απεικονίζει, οδηγώντας το HDMI, μονίμως στην οθόνη. Η προτεινόμενη εικόνα προς εμφάνιση, που μπορεί να χρησιμοποιηθεί και για επαλήθευση της ορθής λειτουργίας του Ελεγκτή, φαίνεται στο Σχήμα 1.



Σχήμα 1: Μια Προτεινόμενη Εικόνα Ελέγχου για Επαλήθευση του HDMI Οδηγού.

Η εικόνα ελέγχου του Σχήματος 1, αποτελείται από επαναλαμβανόμενα μοτίβα οριζόντιων και κατακόρυφων γραμμών των τριών βασικών χρωμάτων απεικόνισης, δηλ. **Κόκκινο**, **Πράσινο** και **Μπλε**. Στο πρώτο μέρος περιλαμβάνει εναλλάξ οριζόντιες γραμμές με κενά, ενώ στο δεύτερο κατακόρυφες γραμμές των τριών χρωμάτων σε φόντο οριζόντιων μαύρων. Η εικόνα ελέγχου θα πρέπει να βρίσκεται αποθηκευμένη στην VRAM για να απεικονιστεί.

Η απεικόνιση της εικόνας ελέγχου στην οθόνη θα πρέπει να είναι σταθερή και ευδιάκριτη. Οι προδιαγραφές οδήγησης της οθόνης είναι:

- Ανάλυση  $640 \times 480$  pixel (*picture element* — στοιχείων οθόνης - κουκίδων) και
- Ρυθμός Ανανέωσης (*refresh rate*) 60Hz.

Λόγω των περιορισμών των συσκευών FPGA σε προσπελάσιμη μνήμη BRAM, το μέγεθος της VRAM, που θα υλοποιηθεί με BRAM θα πρέπει να είναι το 1/5 σε κάθε διάσταση, δηλ.  $128 \times 96$ , και να υποστηρίζει οκτώ χρώματα. Η VRAM θα πρέπει να μεγενθύνεται από τον Ελεγκτή, και να γεμίζει οθόνη ανάλυσης  $640 \times 480$ .

## 2 Το HDMI

Η διεπαφή **HDMI (High Definition Multimedia Interface)** αποτελεί το σύγχρονο πρότυπο για τη μεταφορά ψηφιακού βίντεο και ήχου, αντικαθιστώντας παλαιότερες αναλογικές τεχνολογίες όπως το **VGA**. Η φυσική σύνδεση HDMI χρησιμοποιεί 19 ακίδες, μέσω των οποίων μεταδίδονται πλήρως ψηφιακά σήματα υψηλής ταχύτητας. Η μετάδοση του βίντεο γίνεται με το σχήμα μετάδοσης **TMDS (Transition Minimized Differential Signaling)**, το οποίο βασίζεται σε διαφορικά ζεύγη γραμμών υψηλής συχνότητας. Τα σήματα εικόνας στο HDMI οργάνωνονται σε τέσσερα διαφορικά κανάλια TMDS :

- Τρία κανάλια δεδομένων (TMDS Data 0, 1, 2)
  - καθένα αντιστοιχεί σε μία χρωματική συνιστώσα: Blue, Green, Red
  - σε αντίθεση με το πρότυπο VGA, οι τιμές των χρωμάτων είναι ψηφιακές, συνήθως 8 bit ανά κανάλι (24-bit χρώμα, 16.7 εκατ. χρώματα)
- Ένα κανάλι ρολογιού (TMDS Clock )
  - συγχρονίζει τα δεδομένα των τριών καναλιών

### 2.1 Ψηφιακή Αντιστοίχιση Χρωμάτων

Ο παρακάτω Πίνακας παρουσιάζει μία βασική ψηφιακή χρωματική αντιστοίχιση 8 χρωμάτων, ως προς τις τιμές των σημάτων **Red**, **Green**, **Blue**.

| Κόκκινο<br>(Red) | Πράσινο<br>(Green) | Μπλε<br>(Blue) | Συνισταμένη<br>Χρώματος |
|------------------|--------------------|----------------|-------------------------|
| 0                | 0                  | 0              | Μαύρο                   |
| 0                | 0                  | 1              | Μπλε                    |
| 0                | 1                  | 0              | Πράσινο                 |
| 0                | 1                  | 1              | Κυανό                   |
| 1                | 0                  | 0              | Κόκκινο                 |
| 1                | 0                  | 1              | Μοβ                     |
| 1                | 1                  | 0              | Κίτρινο                 |
| 1                | 1                  | 1              | Άσπρο                   |

### 2.2 Χρονισμός του HDMI

Εκτός από τα δεδομένα χρώματος, το HDMI βασίζεται επίσης σε έναν καλά ορισμένο χρονισμό πλαισίου (video timing), ο οποίος καθορίζει με ακρίβεια πότε αρχίζει και τελειώνει η αποτύπωση της εικόνας. Παρά το γεγονός ότι τα σήματα δεν είναι πλέον αναλογικά, η έννοια των περιοχών **active drawing**, **blanking**, και **sync** παραμένει η ίδια με αυτή των παλαιότερων προτύπων, όπως της **VGA**. Στο Σχήμα 2 απεικονίζεται ο τρόπος με τον οποίο διατρέχεται ένα οποιοδήποτε καρέ (video frame), ενώ στο Σχήμα 3 παρουσιάζονται οι βασικές περιοχές ενός video frame, οι οποίες ισχύουν για κάθε ανάλυση και συχνότητα ανανέωσης.

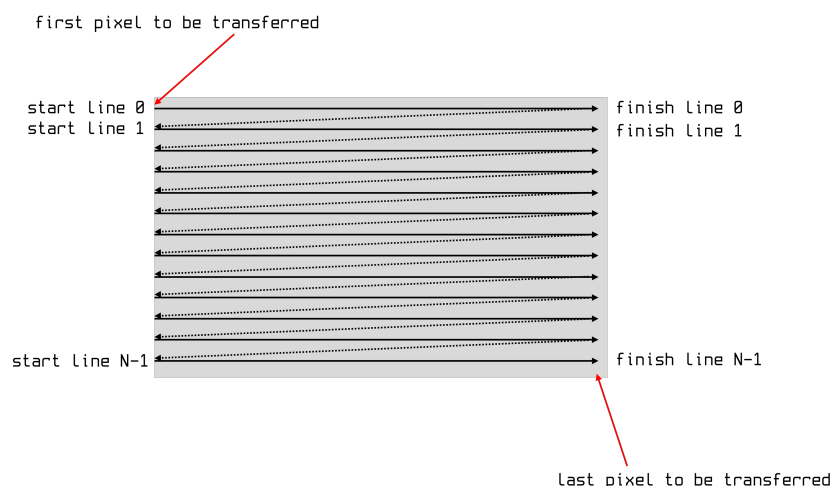
Η περιοχή **active drawing** αντιστοιχεί στα pixel που εμφανίζονται πραγματικά στην οθόνη. Κατά τη διάρκεια αυτής της χρονικής περιόδου, το σύστημα μεταδίδει τις ψηφιακές τιμές των χρωματικών συνιστωσών (R, G, B), οι οποίες καθορίζουν την τελική απεικόνιση.

Εκτός αυτής της περιοχής, η μετάδοση χρωματικών δεδομένων απενεργοποιείται, καθώς η οθόνη δεν βρίσκεται σε φάση εμφάνισης.

Μετά το τέλος κάθε οριζόντιας γραμμής, ακολουθεί η οριζόντια παύση (**horizontal blanking**), η οποία χωρίζεται σε τρία τμήματα: το **horizontal front porch**, το **horizontal sync**, και το **horizontal back porch**. Το **front porch** είναι το μικρό χρονικό διάστημα πριν την έναρξη του παλμού συγχρονισμού, ενώ ο παλμός **hsync** (**horizontal sync**) καθορίζει στην οθόνη ότι η τρέχουσα γραμμή έχει ολοκληρωθεί. Ακολουθεί το **back porch**, το οποίο ολοκληρώνει τη διαδικασία επαναφοράς πριν ξεκινήσει η επόμενη ενεργή γραμμή. Αυτή η ακολουθία επαναλαμβάνεται για κάθε γραμμή του καρέ.

Αντίστοιχη λογική ισχύει και στην κατακόρυφη κατεύθυνση. Μετά το τέλος της τελευταίας ενεργής γραμμής, η οθόνη εισέρχεται στην περιοχή **vertical blanking**, η οποία περιλαμβάνει το **vertical front porch**, το **vertical sync**, και το **vertical back porch**. Ο παλμός **vsync** (**vertical sync**) ενημερώνει την οθόνη ότι ολοκληρώθηκε το τρέχον καρέ και ξεκινά η διαδικασία εμφάνισης του επόμενου. Καθ' όλη τη διάρκεια του **vertical blanking**, η οθόνη δεν εμφανίζει νέα pixel.

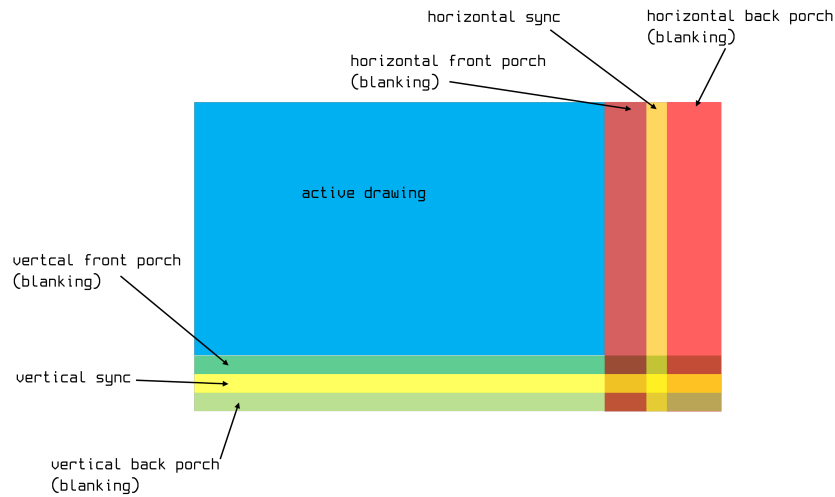
Παρότι οι πραγματικές διάρκειες των περιοχών αυτών διαφέρουν από ανάλυση σε ανάλυση (π.χ. 720p, 1080p), η δομή του χρονισμού παραμένει σταθερή. Η οργάνωση αυτή εξασφαλίζει ότι κάθε καρέ προβάλλεται ομαλά, ότι η οθόνη συγχρονίζεται σωστά με την πηγή, και ότι τα δεδομένα χρώματος αποστέλλονται στον ακριβή χρόνο που απαιτείται.



Σχήμα 2: Απεικόνιση της διάσχισης ενός HDMI Frame.

### 2.2.1 Τιμές Διαστημάτων του Frame

Όπως ειπώθηκε νωρίτερα, ο χρονισμός των διαστημάτων που παρουσιάζονται στο Στο Σχήμα 3 εξαρτάται από την ανάλυση και τον Ρυθμό Ανανέωσης. Ο παρακάτω Πίνακας παρουσιάζει τις κατάλληλες τιμές των διαστημάτων για ανάλυση 640 × 480 και Ρυθμό Ανανέωσης 60Hz. Οι τιμές αυτές αναφέρονται στους κύκλους του ρολογιού που θα εξηγηθεί στην επόμενη υποενότητα. Επιπλέον, στο Σχήμα 4 φαίνονται πώς αυτές οι τιμές αντιστοιχίζονται σε κάθε τμήμα του Frame.



Σχήμα 3: Απεικόνιση των καταστάσεων ενός HDMI Frame.

| Διάστημα               | Διάστημα στα Ελληνικά           | Τιμή |
|------------------------|---------------------------------|------|
| Active Drawing (H)     | Οριζόντια Ενεργή Απεικόνιση     | 640  |
| Horizontal Front Porch | Οριζόντια Μπροστινή Όψη         | 16   |
| Horizontal Sync        | Οριζόντιο Διάστημα Συγχρονισμού | 96   |
| Horizontal Back Porch  | Οριζόντια Πίσω Όψη              | 48   |
| Total Horizontal       | Σύνολο (Οριζόντια)              | 800  |
| Active Drawing (V)     | Κάθετη Ενεργή Απεικόνιση        | 480  |
| Vertical Front Porch   | Κάθετη Μπροστινή Όψη            | 10   |
| Vertical Sync          | Κάθετο Διάστημα Συγχρονισμού    | 2    |
| Vertical Back Porch    | Κάθετη Πίσω Όψη                 | 33   |
| Total Vertical         | Σύνολο (Κάθετα)                 | 525  |

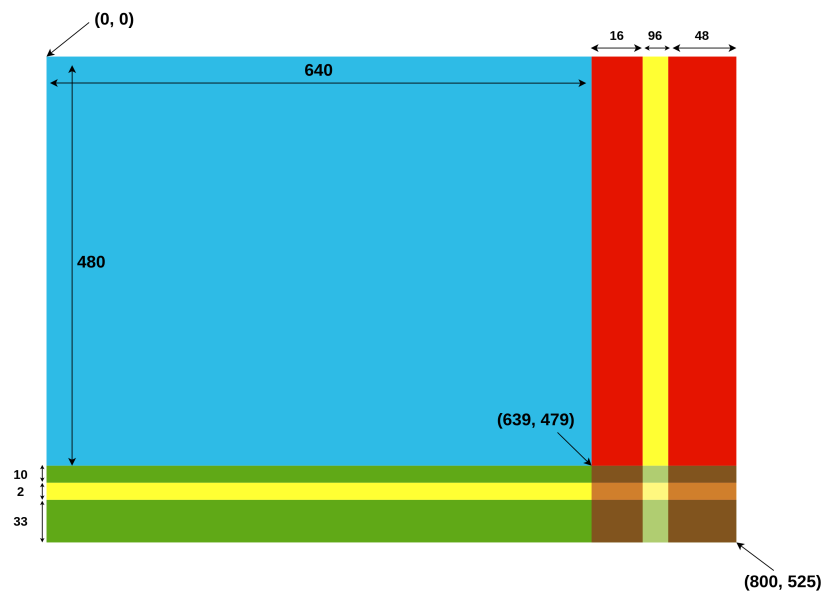


Figure 4: Ο αριθμός κύκλων που αντιστοιχεί σε κάθε τμήμα ενός HDMI Frame.

## 2.3 Χρήση ρολογιού

Η συχνότητα με την οποία αποστέλλονται τα pixels προς την οθόνη καθορίζεται από ένα ρολόι αυστηρά ορισμένης συχνότητας, το οποίο ονομάζεται **pixel clock**. Κάθε παλμός του ρολογιού αντιστοιχεί στη μετάδοση ενός pixel, και συνεπώς η τιμή του καθορίζει τον συνολικό ρυθμό ροής της εικόνας. Η συχνότητα αυτή εξαρτάται από την ανάλυση, τον ρυθμό ανανέωσης, καθώς και από τα χρονικά διαστήματα οριζόντιας και κατακόρυφης απομάκρυνσης που περιγράφηκαν παραπάνω.

Για την ανάλυση  $640 \times 480$  στα 60 Hz, το καθιερωμένο pixel clock είναι 25.175 MHz, σύμφωνα με το πρότυπο VESA. Το ρολόι αυτό δεν χρησιμοποιείται μόνο εσωτερικά για την οδήγηση των κατάλληλων σημάτων και καθορισμό των χρονικών διαστημάτων, αλλά αποτελεί και μία από τις εξόδους (outputs) του κυκλώματος.

## 2.4 Video RAM (VRAM)

Η VRAM αποτελεί βασικό εσωτερικό τμήμα ενός Ελεγκτή/Οδηγού HDMI, μιας και είναι η μνήμη που μονίμως απεικονίζεται την οθόνη. Το μέγεθος της VRAM είναι συνάρτηση της ανάλυσης, μιας και αυτή ορίζει τον αριθμό των εμφανιζόμενων pixels, και του αριθμού των χρωμάτων ανά pixel. Έτσι, η ανάλυση  $640 \times 480$ , απαιτεί  $640 \times 480 = 307200$  bits μνήμης, για κάθε μονόχρωμο pixel. Για τα τρία χρώματα που επιτρέπει η FPGA ανά pixel, απαιτούνται 3 bits, άρα η συνολική μνήμη που χρειάζεται για έγχρωμο pixel είναι  $307200 \times 3 = 115.2$  KBytes μνήμης.

Οι περισσότερες FPGA περιλαμβάνουν εσωτερική RAM (Block RAM - BRAM), η οποία μπορεί να χρησιμοποιηθεί είτε σαν μνήμη 1-bit, είτε σαν μνήμη περισσότερων από 1-bit, και μπορεί να αντιστοιχηθεί, στην υλοποίηση του Ελεγκτή HDMI σε VRAM. Παρόλο που οι FPGA έχουν διαθέσιμη μνήμη BRAM, λ.χ. η **Boolean Board (XC7S50)** έχει 337 KBytes, δεν είναι όλη αυτή η μνήμη προσπελάσιμη ως ένα μονολιθικό τμήμα. Οι διαθέσιμες αντιστοιχίσεις είναι  $16K \times 1$  (16K bits),  $8K \times 2$ ,  $4K \times 4$ ,  $2K \times 8$ ,  $1K \times 16$ , κτλ.

Έτσι, λόγω έλλειψης εύκολα προσπελάσιμης μνήμης BRAM στη συσκευή, και για λόγους απλότητας της υλοποίησης, σας προτείνεται να χρησιμοποιήσετε τρεις BRAM, μια για κάθε χρώμα, και μεγέθους  $16K \times 1$ , για να υλοποιήσετε την απαιτούμενη VRAM  $128 \times 96$ .

## 2.5 Κωδικοποίηση TMDS

Στο HDMI, τα δεδομένα εικόνας μεταφέρονται μέσω καλωδίων με εξαιρετικά υψηλούς ρυθμούς. Για παράδειγμα, η ανάλυση 720p απαιτεί τη μεταφορά 74,25 εκατομμυρίων πιξελ ανά δευτερόλεπτο. Η συνεχής μετάβαση μεταξύ 0 και 1 σε τόσο υψηλές συχνότητες προκαλεί σημαντικό ηλεκτρικό θόρυβο, ο οποίος μπορεί να επηρεάσει αρνητικά την ποιότητα του σήματος – π.χ. πληροφορία από το κόκκινο κανάλι μπορεί να παρεμβληθεί σε εκείνη του μπλε καναλιού.

Για να περιοριστούν οι υψηλές συχνότητας μεταβάσεις των bit και να περιοριστεί ο συνολικός θόρυβος, το πρότυπο HDMI χρησιμοποιεί μια τεχνική κωδικοποίησης που ονομάζεται TMDS (Transition Minimized Differential Signaling). Με αυτή τη μέθοδο, το σύστημα μειώνει τον αριθμό των μεταβάσεων (transitions) μέσα σε μια ακολουθία δεδομένων.

Η συγκεκριμένη τεχνική χρησιμοποιεί πύλες XOR / XNOR για την κωδικοποίηση, και λαμβάνοντας ως είσοδο μία ακολουθία 8-bit, παράγει μία κωδικοποιημένη έξοδο μεγέθους 10-bit. Ο αλγόριθμος απεικονίζεται αναλυτικά στο Σχήμα 5.

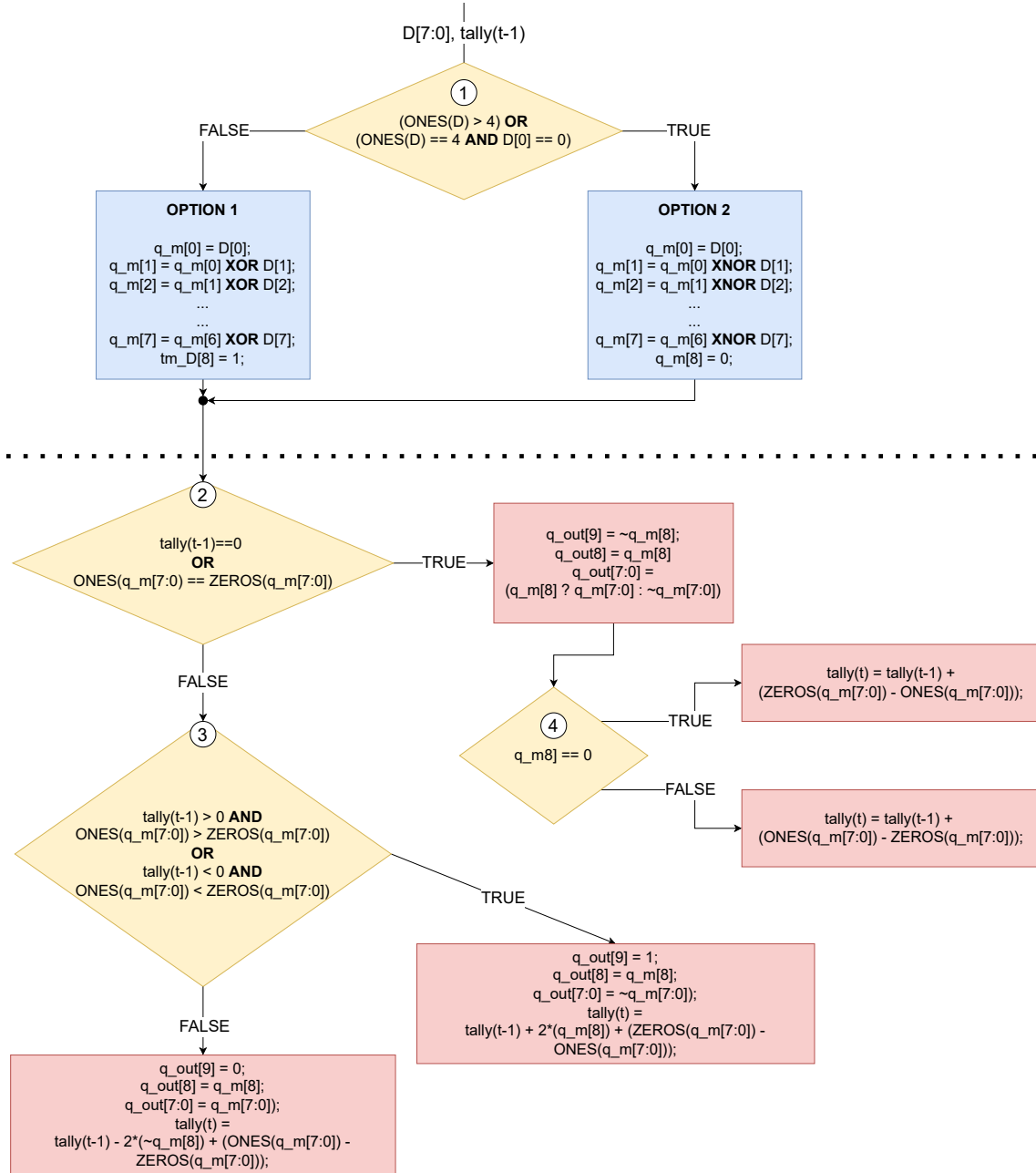


Figure 5: Διάγραμμα του Αλγορίθμου Κωδικοποίησης TMDS.

## 2.6 Σειριοποίηση (Serialization)

Μετά την κωδικοποίηση TMDS, τα 10-bit σύμβολα πρέπει να σταλούν στη γραμμή HDMI σειριακά. Αν και η FPGA παράγει τα δεδομένα παράλληλα (10 bit ανά κύκλο του pixel clock), το φυσικό κανάλι HDMI μεταδίδει ένα bit κάθε φορά. Για τον λόγο αυτό απαιτείται η διαδικασία της σειριοποίησης.

Σε FPGA υλοποιήσεις, η σειριοποίηση πραγματοποιείται μέσω ειδικών hardware blocks, όπως είναι τα USERDES (Output Serializer / Deserializer). Ο serializer λαμβάνει τα 10 bit παράλληλων δεδομένων από τον κωδικοποιητή TMDS και τα εκπέμπει στη γραμμή με συχνότητα δέκα φορές μεγαλύτερη από το pixel clock.

Στην πράξη, μπορεί να χρησιμοποιηθεί ένα ρολόι το οποίο είναι πέντε φορές γρηγορότερο από το pixel clock. Ο Serializer μπορεί να χρησιμοποιήσει και τις δύο ακμές αυτού



του ρολογιού για να μεταδώσει τα δεδομένα με δέκα φορές μεγαλύτερη συχνότητα.

Τέλος, η έξοδος μεταδίδεται διαφορικά μέσω ζεύγους γραμμών (TMDS+ και TMDS-). Το **διαφορικό σήμα** μειώνει τον ηλεκτρομαγνητικό θόρυβο και επιτρέπει αξιόπιστη μετάδοση σε υψηλές ταχύτητες. Αυτό επιτυγχάνεται μέσω **OBUFDS blocks** (Output Buffers generating Differential Signals), οι οποίοι χρησιμοποιούνται τόσο για τα τρία χρωματικά κανάλια, όσο και για το `pixel clock`. Μετά από την παραπάνω διαδικασία, τα σήματα εξόδου του συστήματος είναι τα εξής:

- `hdmi_tx_p[2:0]`: Το θετικό τμήμα του διαφορικού σήματος για καθένα από τα τρία χρωματικά κανάλια
- `hdmi_tx_n[2:0]`: Το αρνητικό τμήμα του διαφορικού σήματος για καθένα από τα τρία χρωματικά κανάλια
- `hdmi_clk_p[2:0]`: Το θετικό τμήμα του διαφορικού σήματος για το HDMI clock
- `hdmi_clk_n[2:0]`: Το αρνητικό τμήμα του διαφορικού σήματος για το HDMI clock

### 3 Δομή της Μονάδας Ελεγκτή/Οδηγού HDMI

Η προτεινόμενη μορφή του Ελεγκτή HDMI φαίνεται στο Σχήμα 6.

```
module hdmi_controller(  
    input clk,  
    input rst,  
    output [2:0] hdmi_tx_p,  
    output [2:0] hdmi_tx_n,  
    output hdmi_clk_p,  
    output hdmi_clk_n  
);  
  
    ...  
  
endmodule
```

Figure 6: Μια Μορφή Υλοποίησης του Ελεγκτή HDMI.

### 4 Μέρος Α - Υλοποίηση VRAM

Η VRAM αποτελεί κεντρικό τμήμα του Ελεγκτή HDMI. Χρησιμοποιώντας τα κατάλληλα πρότυπα BRAM υλοποιήστε την απαιτούμενη VRAM του ελεγκτή. Τα πρότυπα μπορείτε να τα βρείτε, είτε στο σχετικό Τεχνικό Δελτίο, είτε μέσω του Vivado, στο μενού `Tool→Language Templates→Verilog→Device Primitive Instantiation→Artix-7`. Όπως ειπώθηκε νωρίτερα, η VRAM θα πρέπει να χωράει εικόνα μεγέθους  $128 \times 96$  pixel και οκτώ χρωμάτων. Η είσοδος της VRAM θα είναι η διεύθυνση ενός pixel, ενώ η έξοδος της θα παράγει τις τιμές των χρωμάτων του συγκεκριμένου pixel.

Η αρχικοποίηση των BRAM μπορεί να γίνει με πολλούς τρόπους, λ.χ. μέσω παραμέτρων `.INIT_xx`. Έτσι, μπορείτε να περιγράψετε, μέσω των παραμέτρων αυτών, την οθόνη ελέγχου για τη δοκιμή της ορθής υλοποίησης του Ελεγκτή. Για τη δοκιμή της VRAM, γράψτε ένα

απλό πλαίσιο δοκιμής που να διαβάζει pixel από διαφορετικές διευθύνσεις, και με διαφορετικά χρώματα. Η δοκιμή του Μέρους Α αρκεί να γίνει σε επίπεδο συμπεριφοράς.

Όταν η κυκλωματική υλοποίηση είναι ορθή, επιδείξτε τον κώδικα Verilog που γράψατε για το κύκλωμα (RTL Design) και το πλαίσιο δοκιμής (Testbench) και τα αποτελέσματα της προσομοίωσης σε επιτηρητή του εργαστηρίου.

## 5 Μέρος Β - Υλοποίηση Οριζόντιου και Κάθετου μετρητή Pixel και οδήγηση HSYNC, VSYNC, και ACTIVEDRAW

Οδηγήστε τα σήματα `activedraw`, `hsync` (horizontal sync) και `vsync` (vertical sync), με τον κατάλληλο χρονισμό, για ανάλυση  $640 \times 480$  και Ρυθμό Ανανέωσης 60Hz, σύμφωνα με τα Σχήματα 3 και 4 και με τις τιμές του Πίνακα. Σας προτείνεται να χρησιμοποιήσετε έναν οριζόντιο μετρητή `hcount` και έναν κάθετο μετρητή `vcount`, οι οποίοι θα διατρέχουν το σύνολο των ορατών και μη ορατών pixels για κάθε frame. Το σήμα `activedraw` θα είναι ενεργό όταν και οι δύο μετρητές βρίσκονται εντός του συγκεκριμένου τμήματος, ενώ τα σήματα `hsync` και `vsync` θα ενεργοποιούνται όταν ο οριζόντιος μετρητής βρίσκεται στο τμήμα `horizontal sync`, και ο κάθετος στο τμήμα `vertical sync` αντίστοιχα, και θα παραμένουν ανενεργά οπουδήποτε αλλού. Τα σήματα αυτά είναι απαραίτητα να οδηγούνται σωστά, καθώς θα τροφοδοτηθούν στη συνέχεια στους κωδικοποιητές.

Επιπλέον, οδηγήστε ένα σήμα `rgb` μήκους 3bit το οποίο θα αντιπροσωπεύει το χρώμα που βγαίνει από την VRAM σε κάθε χρονική στιγμή, ανάλογα με τη διεύθυνση που δίνεται. Η διεύθυνση pixel της VRAM, θα μπορεί να προκύπτει με τη συνένωση των κατακόρυφων και οριζόντιων μετρητών, δηλ. `address = {vcount, hcount}`, όπου η διεύθυνση θα πρέπει να αποτελείται από τον κατάλληλο αριθμό bits.

Επιβεβαιώστε, μέσω προσομοίωσης, ότι ο χρονισμός είναι σωστός και ότι τα σήματα ενεργοποιούνται κατάλληλα, καθώς και ότι το εκάστοτε ενεργό pixel της VRAM είναι το σωστό σύμφωνα με την προτεινόμενη εικόνα.

Για το απαιτούμενο pixel clock χρειάζεται να γίνει χρήση μίας μονάδας Mixed Mode Clock Manager (MMCM) ώστε να παράγεται ένα ρολόι συχνότητας 25.175MHz από το ρολόι της FPGA. Το ρολόι αυτό θα χρησιμοποιηθεί σε όλα τα επιμέρους κυκλώματα του ελεγκτή. Προτείνεται, κάνοντας χρήση της ίδιας μονάδας, να κατασκευασετε και ένα ρολόι συχνότητας 125.875 MHz, δηλαδή 5 φορές πιο γρήγορο, επειδή θα χρησιμοποιηθεί στα επόμενα βήματα.

Μία προτεινόμενη υλοποίηση του συστήματος απεικονίζεται στο Σχήμα 7.

Όταν η κυκλωματική υλοποίηση είναι ορθή, επιδείξτε τον κώδικα Verilog που γράψατε για το κύκλωμα (RTL Design) και το πλαίσιο δοκιμής (Testbench) και τα αποτελέσματα της προσομοίωσης σε επιτηρητή του εργαστηρίου.

## 6 Μέρος Γ - Κωδικοποίηση TMDS

Για την υλοποίηση του Μέρους Γ, είναι απαραίτητο να ακολουθήσετε λεπτομερώς και με πολλή προσοχή το διάγραμμα του Σχήματος 5. Στα παρακάτω βήματα περιγράφεται ο αλγόριθμος, χωριζόμενος για ευκολία σε δύο τμήματα, όπως υποδεικνύεται από τη διακεκομ-

```

module timing_and_color_driver(
    input clk,
    input rst,
    output activedraw,
    output hsync,
    output vsync,
    output [2:0] rgb // 1bit each for now
);

// include MMCM, VRAM, FSM(s), counter(s), etc //

...

endmodule

```

Figure 7: Μια Μορφή Υλοποίησης του Μέρους B.

μένη γραμμή. Σας προτείνεται να υλοποιήσετε και να ελέγξετε το Μέρος Γ ανεξάρτητα από τα προηγούμενα.

## 6.1 Ελαχιστοποίηση μεταβάσεων (Transition Minimization)

Σκοπός της κωδικοποίησης είναι να ελαχιστοποιηθεί ο αριθμός των αλλαγών μεταξύ 0 και 1 κατά την κωδικοποίηση της δυαδικής συμβολοσειράς που αντιπροσωπεύει το χρώμα ενός pixel στο κάθε κανάλι, ώστε να βελτιστοποιηθεί η ταχύτητα μετάδοσης των δεδομένων, που είναι απαραίτητο για σήματα high-speed. Υπάρχουν δύο περιπτώσεις κωδικοποίησης, και για κάθε δυαδική ακολουθία επιλέγεται η κωδικοποίηση που επιφέρει τις λιγότερες μεταβάσεις μεταξύ λογικού 0 και 1. Είσοδος στο σύστημα είναι μία δυαδική ακολουθία μεγέθους 8 bit, έστω  $x$ , και έξοδος μία δυαδική ακολουθία μεγέθους 9 bit, έστω  $y$ . Η ελαχιστοποίηση των αλλαγών πραγματοποιείται με τη χρήση πυλών XOR (Περίπτωση 1) ή XNOR (Περίπτωση 2), αναλυτικά:

- Και στις δύο περιπτώσεις, το LSB της εξόδου είναι ίδιο με το LSB της εισόδου.
- Τα επόμενα 7 bits καθορίζονται:
  - (Περίπτωση 1) Από την εξίσωση  $y_n = x_n \oplus y_{n-1}$
  - (Περίπτωση 2) Από την εξίσωση  $y_n = \overline{x_n \oplus y_{n-1}}$
- Για το MSB:
  - (Περίπτωση 1) Ίσο με 1
  - (Περίπτωση 2) Ίσο με 0

Από τις δύο περιπτώσεις επιλέγουμε εκείνη με τον μικρότερο αριθμό μεταβάσεων, ωστόσο δεν χρειάζεται κάθε φορά να ελέγχονται και οι δύο περιπτώσεις, καθώς έχει αποδειχθεί ότι η συνθήκη για την επιλογή της βέλτιστης περίπτωσης είναι η εξής:

- Αν το άθροισμα των 1 στην είσοδο είναι μεγαλύτερο του 4 ή
- Αν το άθροισμα των 1 στην είσοδο είναι 4 και το LSB είναι 0

Τότε επιλέγεται η Περίπτωση 2. Αν δεν ισχύει το παραπάνω, επιλέγεται η Περίπτωση 1. Για παράδειγμα, για τη σειρά 8b'1111\_1110 η Περίπτωση 1 δίνει αποτέλεσμα 9'b1\_1010\_1010 ενώ η Περίπτωση 2 δίνει αποτέλεσμα 9'b0\_0000\_0000. Μία προτεινόμενη υλοποίηση του συστήματος απεικονίζεται στο Σχήμα 8. Αξίζει να σημειωθεί ότι το συγκεκριμένο κύκλωμα είναι εξ' ολοκλήρου συνδυαστικό, καθώς θέλουμε να αποφύγουμε την καθυστέρηση που επιφέρει η ακολουθιακή λογική.

```
module tm_choice (
    input [7:0] data_in,
    output reg [8:0] q_m
);

...

endmodule
```

Figure 8: Μια Μορφή Υλοποίησης του κυκλώματος Transition Minimization.

## 6.2 Ολοκλήρωση κωδικοποίησης TMDS

Ο σκοπός αυτού του τμήματος της κωδικοποίησης είναι η επίτευξη ισορροπίας μεταξύ των 1 και 0 που μεταδίδονται. Αυτό είναι αναγκαίο να γίνει για να αποφευχθούν προβλήματα σχετικά με την DC τάση.

Σε αντίθεση με το προηγούμενο μέρος, το συγκεκριμένο δουλεύει σε επίπεδο κύκλων, και σε κάθε κύκλο ανανεώνει την πληροφορία που κρατάει σε σχέση με τον αριθμό 1 και 0 που έχουν αποσταλλεί πρόσφατα. Αυτή η πληροφορία αποθηκεύεται στην προσημασμένη μεταβλητή **tally**. Εάν το **tally** έχει θετική τιμή, σημαίνει πως τα πρόσφατα 1 ήταν περισσότερα από τα 0, ενώ αν έχει αρνητική τιμή συμβαίνει το αντίθετο. Σε κάθε περίπτωση, ο σκοπός του αλγορίθμου είναι να στέλνει 1 ή 0 ανάλογα, ώστε το **tally** να μην απομακρύνεται πολύ από την ισορροπία (**tally** = 0).

Στην πράξη, η μεταβλητή **tally** χρειάζεται να αποθηκεύει τους αριθμούς από το -10 ως το 10, γιατί ο μέγιστος αριθμός 1 ή 0 που μπορεί να έρθει σαν είσοδος οποιαδήποτε στιγμή είναι το 10. Αυτό μπορεί να γίνει με έναν καταχωρητή 5 bits, όπου το MSB αντιπροσωπεύει το πρόσημο, και τα υπόλοιπα 4 bits περιέχουν την απόλυτη τιμή. Έτσι, όταν γίνεται έλεγχος για την τιμή του **tally**, μπορεί να ελέγχεται το πρόσημο ξεχωριστά.

Για παράδειγμα, έστω ότι η έξοδος του **tm\_choice** είναι 9'b1\_0000\_0101, και το τρέχον **tally** είναι -2, σύμφωνα με το διάγραμμα στο Σχήμα 8:

- (Συνθήκη 2) Το **tally** δεν είναι 0, και ο αριθμός των 0 δεν είναι ίσος με τον αριθμό των 1. Επομένως ακολουθούμε την επιλογή "False".
- (Συνθήκη 3) Βλέπουμε ότι το **tally** είναι < 0 και ότι τα 0 είναι περισσότερα από τα 1, άρα ακολουθούμε την επιλογή "True".
- Θέτουμε το MSB στο 1, κρατάμε ίδιο το bit 8, και αντιστρέφουμε όλα τα άλλα. Επομένως, το μήνυμα θα είναι 10'b11\_1111\_1010.
- Ενημερώνουμε το **tally** προσθέτοντας 2 + τη διαφορά 1 με 0 του αρχικού μηνύματος, η οποία είναι 6 στην περίπτωση μας. Άρα, **tally** = -2 + 2 + 6 = +6

Κάποιες επιπλέον περιπτώσεις στις οποίες θα πρέπει να δοθεί προσοχή:

- Όταν το σύστημα γίνεται **reset**, η έξοδος θα πρέπει να είναι ακολουθία από 0, και το **tally** θα πρέπει να είναι 0.
- Όταν η είσοδος **ve\_in** (**video\_enable\_in**) είναι ενεργοποιημένη, η συμπεριφορά είναι αυτή που περιγράφεται παραπάνω. Όταν όμως δεν είναι, το **tally** μηδενίζεται και η έξοδος εξαρτάται από την είσοδο **control\_in** ως εξής:
  - 2'b00: 10'b1101010100
  - 2'b01: 10'b0010101011
  - 2'b10: 10'b0101010100
  - 2'b11: 10'b1010101011

Για διευκόλυνση στην επαλήθευση, σας δίνεται ένα **testbench** που επιβεβαιώνει τη σωστή λειτουργία του κυκλώματος δοκιμάζοντας πολλαπλές διαδοχικές ακολουθίες εισόδου. Είναι σημαντικό, για κάθε ακολουθία bits της εισόδου, όλες οι λογικές πράξεις να πραγματοποιούνται σε έναν μόνο κύκλο του ρολογιού.

Μία προτεινόμενη υλοποίηση του συστήματος απεικονίζεται στο Σχήμα 9.

```
module tmds_encoder(  
    input clk,  
    input rst,  
    input [7:0] data_in,  
    input [1:0] control_in,  
    input ve_in, // video data enable  
    output [9:0] tmds_out  
);  
  
wire [8:0] q_m;  
  
tm_choice tm_choice_inst(  
    .data_in(data_in),  
    .q_m(q_m)  
);  
  
...  
  
endmodule
```

Figure 9: Μια Μορφή Υλοποίησης του κυκλώματος TMDS Encoder.

Όταν η κυκλωματική υλοποίηση είναι ορθή, επιδείξτε τον κώδικα Verilog που γράψατε για το κύκλωμα (RTL Design) και το πλαίσιο δοκιμής (Testbench) και τα αποτελέσματα της προσομοίωσης σε επιτηρητή του εργαστηρίου.

## 7 Μέρος Δ - Ολοκλήρωση του Ελεγκτή/Οδηγού HDMI

Σε αυτό το βήμα θα ενώσετε τα επιμέρους κυκλώματα που κατασκευάσατε στα προηγούμενα μέρη της εργασίας, έτσι ώστε να επιτευχθεί η οδήγηση της οθόνης.

- Θα χρησιμοποιηθεί ένας κωδικοποιητής για κάθε χρώμα. Η αρχικοποίηση των τριών TMDs Encoders παρουσιάζεται στο Σχήμα 10. Όπως φαίνεται, το σήμα `activedraw` είναι απαραίτητο και για τα τρία χρώματα, ενώ τα σήματα `hsync`, `vsync` είναι απαραίτητα μόνο για το μπλε χρώμα, ώστε να παράγονται κατάλληλα οι ακολουθίες που μεταδίδουν την πληροφορία του `blanking` και του `sync`.
- Η είσοδος των encoders που αντιστοιχεί στο εκάστοτε χρώμα, θα έχει μέγεθος 8 bit, και θα προκύπτει από την χρήση του αντίστοιχου bit που προκύπτει από την VRAM, 8 φορές. Έτσι, αν το χρώμα που προκύπτει από την VRAM είναι, π.χ., το κόκκινο (3'b100), θα πρέπει οι ακολουθίες των 8-bit που θα εισέρχονται στους τρεις κωδικοποιητές να είναι οι εξής:

- Red: 8'b1111\_1111
- Green: 8'b0000\_0000
- Blue: 8'b0000\_0000

```

tmds_encoder tmds_red(
    .clk_in(clk_pixel), // your pixel clock
    .rst_in(rst),       // system reset
    .data_in(red),      // 8-bit value
    .control_in(2'b0),
    .ve_in(activedraw), // from part2
    .tmds_out(tmds_10b[2]));

tmds_encoder tmds_green(
    .clk_in(clk_pixel), // your pixel clock
    .rst_in(rst),       // system reset
    .data_in(green),    // 8-bit value
    .control_in(2'b0),
    .ve_in(activedraw), // from part2
    .tmds_out(tmds_10b[1]));

tmds_encoder tmds_blue(
    .clk_in(clk_pixel), // your pixel clock
    .rst_in(rst),       // system reset
    .data_in(blue),     // 8-bit value
    .control_in({vsync, hsync}), // from part2
    .ve_in(activedraw), // from part2
    .tmds_out(tmds_10b[0]));

```

Figure 10: Αρχικοποίηση των τριών εμφανίσεων του TMDs Encoder.

- Η αρχικοποίηση των τριών μονάδων TMDs Serializer και των τριών μονάδων OBUFDS παρουσιάζεται στο Σχήμα 11. Το ρολόι συχνότητας 125.875 MHz που έχετε ήδη δημιουργήσει μέσω του MMCM χρησιμοποιείται σε αυτό το στάδιο, ώστε να επιτευχθεί η επιθυμητή ταχύτητα του Serialization. Το κύκλωμα του TMDs Serializer επισυνάπτεται μαζί με την εκφώνηση της εργασίας.

```

tmlds_serializer red_ser(
    .clk_pixel_in(clk_pixel), // your pixel clock
    .clk_5x_in(clk_5x),      // your x5 clock
    .rst_in(rst),            // system reset
    .tmlds_in(tmlds_10b[2]),
    .tmlds_out(tmlds_signal[2]));

tmlds_serializer green_ser(
    .clk_pixel_in(clk_pixel), // your pixel clock
    .clk_5x_in(clk_5x),      // your x5 clock
    .rst_in(rst),            // system reset
    .tmlds_in(tmlds_10b[1]),
    .tmlds_out(tmlds_signal[1]));

tmlds_serializer blue_ser(
    .clk_pixel_in(clk_pixel), // your pixel clock
    .clk_5x_in(clk_5x),      // your x5 clock
    .rst_in(rst),            // system reset
    .tmlds_in(tmlds_10b[0]),
    .tmlds_out(tmlds_signal[0]));

OBUFDS OBUFDS_blue (.I(tmlds_signal[0]), .O(hdmi_tx_p[0]), .OB(hdmi_tx_n[0]));
OBUFDS OBUFDS_green(.I(tmlds_signal[1]), .O(hdmi_tx_p[1]), .OB(hdmi_tx_n[1]));
OBUFDS OBUFDS_red  (.I(tmlds_signal[2]), .O(hdmi_tx_p[2]), .OB(hdmi_tx_n[2]));

OBUFDS OBUFDS_clock(.I(clk_pixel), .O(hdmi_clk_p), .OB(hdmi_clk_n));

```

Figure 11: Αρχικοποίηση των τριών εμφανίσεων του TMDS Serializer και των τεσσάρων εμφανίσεων του OBUFDS.

## 7.1 Περιορισμοί XDC

Επιπλέον, χρησιμοποιήστε τις εντολές που φαίνονται στο Σχήμα 12 για τη διαμόρφωση των constraints. Είναι απαραίτητο τα διαφορικά σήματα εξόδου να οριστούν με το IOSTANDARD TMDS\_33 αντί του LVCMOS33, ώστε να μπορεί να επιτευχθεί η υψηλή τους ταχύτητα.

## 7.2 Προχωρημένη Χρήση του Ελεγκτή HDMI (Προαιρετικό)

Προαιρετικά, δοκιμάστε να αποτυπώσετε περισσότερα από 8 χρώματα, κάνοντας χρήση και των 8 bits των χρωματικών καναλιών του HDMI. Αξιοποιήστε τα Switches της πλακέτας για να καθορίζετε ποια από τα 8 bits του κάθε καναλιού θα αποκτούν την τιμή τους μέσω της VRAM, και ποια θα έχουν προκαθορισμένη τιμή (0 ή 1). Δοκιμάστε, με αυτόν τον τρόπο, να εμφανίσετε στην οθόνη τη ζητούμενη εικόνα με διαφορετική ένταση των επιμέρους χρωμάτων, και καταγράψτε στην εργαστηριακή αναφορά τις παρατηρήσεις σας.

(Θα δοθεί *bonus* στην βαθμολογία για την υλοποίηση του προαιρετικού μέρους)

Όταν η κυκλωματική υλοποίηση είναι ορθή, επιδείξτε τον κώδικα Verilog που γράψατε για το κύκλωμα (RTL Design) και το πλαίσιο δοκιμής (Testbench), τα αποτελέσματα της προσομοίωσης, και το κύκλωμα εν λειτουργία σε επιτηρητή του εργαστηρίου.

```

create_clock -period 10.000 -name clk [get_ports clk]
set_property -dict {PACKAGE_PIN F14 IOSTANDARD LVCMOS33} [get_ports {clk}]

# On-board Buttons
set_property -dict {PACKAGE_PIN J2 IOSTANDARD LVCMOS33} [get_ports {rst}]

#HDMI Signals
set_property -dict { PACKAGE_PIN T14 IOSTANDARD TMDS_33} [get_ports {hdmi_clk_n}]
set_property -dict { PACKAGE_PIN R14 IOSTANDARD TMDS_33} [get_ports {hdmi_clk_p}]

set_property -dict { PACKAGE_PIN T15 IOSTANDARD TMDS_33} [get_ports {hdmi_tx_n[0]}]
set_property -dict { PACKAGE_PIN R17 IOSTANDARD TMDS_33} [get_ports {hdmi_tx_n[1]}]
set_property -dict { PACKAGE_PIN P16 IOSTANDARD TMDS_33} [get_ports {hdmi_tx_n[2]}]

set_property -dict { PACKAGE_PIN R15 IOSTANDARD TMDS_33} [get_ports {hdmi_tx_p[0]}]
set_property -dict { PACKAGE_PIN R16 IOSTANDARD TMDS_33} [get_ports {hdmi_tx_p[1]}]
set_property -dict { PACKAGE_PIN N15 IOSTANDARD TMDS_33} [get_ports {hdmi_tx_p[2]}]

# Set Bank 0 voltage
set_property CFGBVS VCC0 [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]

```

Figure 12: Το αρχείο XDC (Xilinx Design Constraints) που θα χρησιμοποιηθεί.

## 8 Προθεσμία Παράδοσης, Υποβολή της Εργασίας και Αναφορά

Η προθεσμία παράδοσης της 3<sup>ης</sup> εργασίας είναι η **19/12/2025**.

Μέχρι την προθεσμία της εργασίας θα πρέπει:

- να έχετε επιδείξει όλα τα επιμέρους μέρη της εργασίας στους επιτηρητές,
- να έχετε υποβάλλει τον κώδικα σας (RTL Design και Testbench) για κάθε σκέλος υλοποίησης ξεχωριστά,
- να έχετε υποβάλλει μια εργαστηριακή αναφορά, βάση των σημειώσεων απο το βιβλίο του εργαστηρίου, **με όλη την απαραίτητη πληροφορία** (*Dataflow, FSM State Graph(s), Simulation Screenshot(s), etc.*).

## 9 Ερωτήσεις και Απορίες

Για οποιεσδήποτε ερωτήσεις και απορίες εκμεταλλευτείτε τον ιστότοπο του e-Class και το χρόνο του εργαστηρίου ρωτώντας τους επιτηρητές.